Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh

Khoa Điện Tử Viễn Thông

# BÁO CÁO ĐỒ ÁN

# Thiết kế bộ lọc FIR thông thấp để lọc nhiễu tín hiệu âm thanh

NHÓM 4: GAUSSIAN

| HỌ TÊN THÀNH VIÊN | MSSV |
|---|---|
| Nguyễn Minh Anh | 22207004 |
| Trần Khánh Hiệp | 22207025 |
| Nguyễn Trung Hiếu | 22207027 |
| Võ Minh Huy | 22207042 |

# MỤC LỤC

Nội dung trang

Chương I : Đặt vấn đề ..... 2

Chương II : Lý thuyết ..... 3

1. Các loại bộ lọc ..... 3
   - *a) Bộ lọc thông thấp FIR (Low-pass FIR filter)* ..... 3
   - *b) Bộ lọc thông cao FIR (High-pass FIR filter)* ..... 3
   - *c) Bộ lọc dải thông FIR (Band-pass FIR filter)* ..... 3
   - *d) Bộ lọc dải chặn FIR (Band-stop FIR filter)* ..... 3
2. Sử dụng LPF (Low-pass filter ) cho bộ lọc FIR ..... 4
   - *a) Phương trình tín hiệu vào ra của lọc FIR* ..... 4
   - *b) Đáp ứng tần số cho bộ lọc thông thấp* ..... 4
   - *c) Đáp ứng xung cho bộ lọc thông thấp* ..... 4
   - d) *Các loại cửa sổ được sử dụng trong lọc FIR* ..... 4
   - e) *Mục tiêu hướng đến* ..... 6

Chương III : Thực hiện xử lý tín hiệu âm thanh ..... 7

1. Ý tưởng cho bộ lọc ..... 7
2. Tạo nhiễu cho tín hiệu bằng phân phối Gaussian ..... 7
   - *a) Khái niệm nhiễu Gaussian* ..... 7
   - *b) Công thức tính nhiễu* ..... 7
   - *c) Đoạn code để tạo nhiễu Gaussian cao tần* ..... 8
3. Thiết kế bộ lọc thông thấp FIR ..... 9
   - a) *Đoạn code lọc thông thấp FIR* ..... 9
   - *b) Giải thích lý do chọn cửa sổ Hamming cho thiết kế* ..... 9
4. Đoạn code hoàn chỉnh cho toàn chương trình ..... 10

Chương IV : Kết quả của tín hiệu ..... 13

1. Các phổ tín hiệu của các file âm thanh ..... 13
2. Kết quả tính toán từ đoạn code ..... 15

Chương V : Kết luận ..... 17

# Chương I : Đặt vấn đề

Trong thế giới số hóa ngày nay, tín hiệu là xương sống của mọi công nghệ, từ truyền thông, y tế đến các hệ thống điều khiển tự động. Tuy nhiên, những tín hiệu quý giá này thường không tránh khỏi sự xâm nhập của nhiễu, làm suy giảm chất lượng và độ chính xác của chúng. Một trong những loại nhiễu phổ biến và "đáng ghét" nhất là nhiễu Gaussian, loại nhiễu có đặc tính phân bố theo quy luật phân phối chuẩn, gây ra sự sai lệch và méo mó trong tín hiệu gốc. Việc loại bỏ hoặc giảm thiểu nhiễu Gaussian là một thách thức quan trọng trong lĩnh vực xử lý tín hiệu, đòi hỏi các kỹ thuật lọc hiệu quả.

Nhưng không cần phải lo lắng quá nhiều, bởi vì chúng ta có một vũ khí mạnh mẽ trong kho vũ khí xử lý tín hiệu - lọc FIR (Finite Impulse Response). Lọc FIR không chỉ nổi bật với độ ổn định và khả năng điều chỉnh tuyệt vời mà còn dễ dàng thiết kế để đáp ứng những yêu cầu đặc biệt của từng ứng dụng, từ việc loại bỏ nhiễu tần số cao đến việc làm mượt các biến đổi tín hiệu.

Bạn có bao giờ tự hỏi làm thế nào để giảm thiểu nhiễu Gaussian và khôi phục tín hiệu nguyên gốc? Bạn có muốn tìm hiểu một phương pháp tối ưu để cải thiện chất lượng tín hiệu mà vẫn giữ được các chi tiết quan trọng? Đây chính là những gì bài báo cáo này sẽ mang lại. Chúng tôi sẽ không chỉ giới thiệu lý thuyết cơ bản về nhiễu Gaussian và lọc FIR, mà còn đi sâu vào phân tích và trình bày các ví dụ thực nghiệm để minh họa hiệu quả của phương pháp này.

Hãy cùng khám phá và chứng kiến sự kỳ diệu của lọc FIR trong việc biến những tín hiệu nhiễu trở lại thành những bản gốc hoàn hảo. Qua đó, chúng ta sẽ thấy rõ ràng hơn về tầm quan trọng và ứng dụng rộng rãi của công nghệ lọc số trong cuộc sống hiện đại.

# Chương II : Lý thuyết

## 1. Các loại bộ lọc

Các bộ lọc FIR (Finite Impulse Response) là một loại bộ lọc số, được sử dụng rộng rãi trong xử lý tín hiệu do tính ổn định và dễ thiết kế. Dưới đây là giới thiệu về các loại bộ lọc FIR phổ biến:

*a) Bộ lọc thông thấp FIR (Low-pass FIR filter) :*

- Cho phép các tần số dưới một ngưỡng xác định đi qua và chặn các tần số cao hơn.
- Ứng dụng

- Loại bỏ nhiễu cao tần từ tín hiệu âm thanh, giúp âm thanh trở nên trong trẻo hơn.
- Loại bỏ nhiễu cao tần từ tín hiệu cảm biến trong các hệ thống đo lường và điều khiển.
- Sử dụng trong các thiết bị nghe nhìn để giảm thiểu tạp âm và cải thiện chất lượng hình ảnh và âm thanh.

*b) Bộ lọc thông cao FIR (High-pass FIR filter) :*

- Cho phép các tần số trên một ngưỡng xác định đi qua và chặn các tần số thấp hơn.
- Ứng dụng:

- Loại bỏ nhiễu thấp tần hoặc tạp âm nền từ tín hiệu, như tiếng ồn môi trường trong tín hiệu âm thanh.
- Sử dụng trong xử lý hình ảnh để làm nổi bật các chi tiết cạnh và biên của hình ảnh.
- Loại bỏ các tín hiệu DC và các thành phần tần số thấp không mong muốn từ tín hiệu điện tử.

*c) Bộ lọc dải thông FIR (Band-pass FIR filter) :*

- Cho phép một dải tần số nhất định đi qua và chặn các tần số bên ngoài dải này.
- Ứng dụng:

- Lọc ra các tần số cụ thể từ tín hiệu hỗn hợp, như trong truyền thông vô tuyến để chọn lọc các kênh cụ thể.
- Sử dụng trong các hệ thống phân tích phổ để phân tích các dải tần số cụ thể của tín hiệu.
- Ứng dụng trong âm thanh để tăng cường hoặc giảm bớt một dải tần số nhất định, như điều chỉnh âm sắc của nhạc cụ.

*d) Bộ lọc dải chặn FIR (Band-stop FIR filter) :*

- Chặn một dải tần số nhất định và cho phép các tần số bên ngoài dải này đi qua.
- Ứng dụng:

- Loại bỏ tần số nhiễu cụ thể từ tín hiệu, như nhiễu 50/60Hz từ mạng điện trong các thiết bị y tế hoặc thiết bị đo lường.

- Sử dụng trong hệ thống âm thanh để loại bỏ tiếng ù từ thiết bị điện tử.
- Loại bỏ các tần số không mong muốn trong tín hiệu truyền thông và xử lý hình ảnh.

❖ Đặc điểm của bộ lọc FIR:

– Độ ổn định : Bộ lọc FIR luôn ổn định vì không có phản hồi từ đầu ra về đầu vào. Điều này đảm bảo rằng tín hiệu đầu ra sẽ không bị dao động không mong muốn.
– Đáp ứng xung hữu hạn : Đáp ứng xung của bộ lọc FIR sẽ dừng sau một số mẫu hữu hạn, do đó nó có thể dễ dàng phân tích và thiết kế.
– Đáp ứng tần số có thể điều chỉnh dễ dàng : Với các công cụ thiết kế hiện đại, bộ lọc FIR có thể được thiết kế để có đáp ứng tần số rất chính xác. Người thiết kế có thể xác định chính xác các đặc tính của bộ lọc như tần số cắt, độ dốc của đáp ứng tần số và đáp ứng pha.

## 2. Sử dụng LPF (Low-pass filter ) cho bộ lọc FIR

*a) Phương trình tín hiệu vào ra của lọc FIR :*

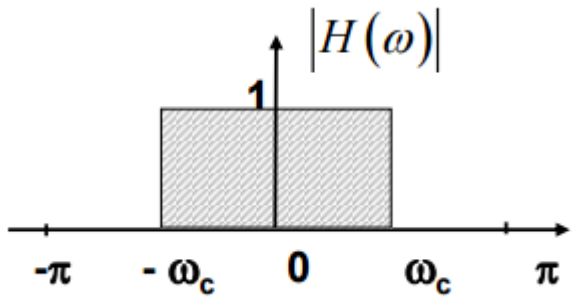
$$y(n) = \sum_{r=0}^{M} b_r x(n-r) = \sum_{r=0}^{M} h(r)x(n-r)$$

❖ Đáp ứng xung :

$h = \{b_0, b_1, b_2, b_3, \ldots, b_M\}$

*b) Đáp ứng tần số cho bộ lọc thông thấp :*

$$|H(\omega)| = \begin{cases} 1, & |\omega| \le \omega_c \\ 0, & \omega_c \le \omega \le \pi \end{cases}$$



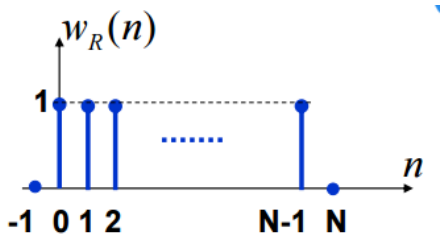

*c) Đáp ứng xung cho bộ lọc thông thấp :*

$$h(n) = \begin{cases} \omega_c/\pi, & n = 0 \\ \dfrac{\sin(\omega_c n)}{\pi n}, & n \ne 0 \end{cases}$$

*d) Các loại cửa sổ được sử dụng trong lọc FIR*

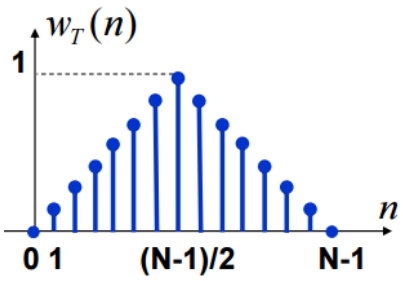
– Cửa sổ hình chữ nhật :

$$w_R(n) = \begin{cases} 1, & 0 \le n \le N-1 \\ 0, & \text{nơi khác} \end{cases}$$



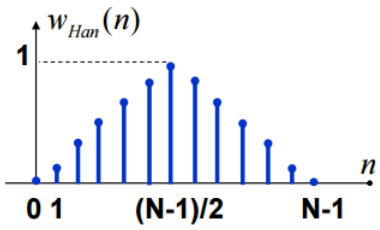

– Cửa sổ hình tam giác :

$$w_T(n) = \begin{cases} \frac{2n}{N-1}, & 0 \le n \le \frac{N-1}{2} \\ 2 - \frac{2n}{N-1}, & \frac{N-1}{2} \le n \le N-1, \\ 0, & \text{nơi khác} \end{cases}$$



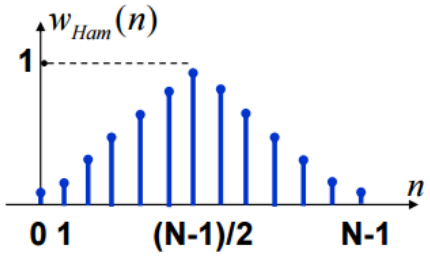

– Cửa sổ Hanning :

$$w_{Han}(n) = \begin{cases} 0.5 - 0.5\cos\left(\frac{2\pi n}{N-1}\right), & 0 \le n \le N-1 \\ 0, & \text{nơi khác} \end{cases}$$



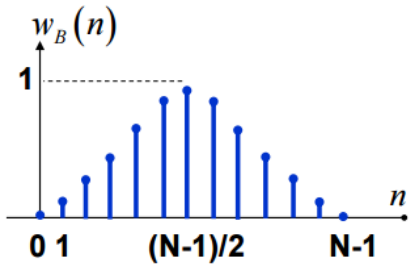

– Cửa sổ Hamming :

$$w_{Ham}(n) = \begin{cases} 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right), & 0 \le n \le N-1 \\ 0, & \text{nơi khác} \end{cases}$$




– Cửa sổ Blackman :

$$w_B(n) = \begin{cases} 0.42 - 0.5\cos\left(\frac{2\pi n}{N-1}\right) + 0.08\cos\left(\frac{4\pi n}{N-1}\right), & 0 \le n \le N-1 \\ 0, & \text{nơi khác} \end{cases}$$




*e) Mục tiêu hướng đến :*

– Lọc thông thấp FIR (Finite Impulse Response) được sử dụng để xử lý tín hiệu âm thanh, giúp cải thiện chất lượng âm thanh bằng cách loại bỏ nhiễu và các thành phần tần số cao không mong muốn.
– Điều này giúp làm mượt tín hiệu và bảo vệ các thiết bị khỏi nhiễu cao tần.
– Bài báo cáo này sẽ minh họa rõ ràng cách hoạt động của bộ lọc thông thấp FIR thông qua việc xử lý tín hiệu âm thanh bị nhiễu từ một file nhạc bất kỳ.
– Qua đó, chúng ta sẽ ứng dụng những kiến thức đã học từ môn xử lý tín hiệu số vào các tình huống thực tế, giúp người đọc có cái nhìn trực quan và hiểu rõ hơn về tác dụng của bộ lọc thông thấp FIR.
– Ưu điểm: Bộ lọc thông thấp FIR có khả năng loại bỏ nhiễu hiệu quả, cải thiện chất lượng tín hiệu và dễ dàng điều chỉnh các tham số lọc theo nhu cầu cụ thể.
– Hạn chế: Quá trình thiết kế và thực hiện bộ lọc FIR có thể phức tạp, yêu cầu tài nguyên tính toán lớn hơn so với một số loại lọc khác, và có thể gây ra độ trễ trong quá trình xử lý tín hiệu.

# Chương III : Thực hiện xử lý tín hiệu âm thanh

## 1. Ý tưởng cho bộ lọc

❖ Bước 1:

– Nhóm chúng em sẽ đưa vào hệ thống xử lý một file âm thanh (một bài nhạc) đã được thu âm trong phòng kín và không chịu ảnh hưởng bởi nhiễu.

❖ Bước 2:

– Tạo nhiễu Gaussian (cao tần) tác động lên toàn bộ bài nhạc trong file âm thanh.

❖ Bước 3:

– Sử dụng bộ lọc thông thấp FIR để lọc nhiễu Gaussian khỏi dữ liệu âm thanh.

❖ Bước 4:

– Xuất ra hai file âm thanh:

- Một file chứa âm thanh có nhiễu Gaussian.
- Một file đã được lọc bằng FIR.

❖ Bước 5:

– Phân tích và so sánh chất lượng âm thanh cùng với phổ tín hiệu của hai file, từ đó đưa ra kết luận cho bài báo cáo.

## 2. Tạo nhiễu cho tín hiệu bằng phân phối Gaussian

*a) Khái niệm nhiễu Gaussian*

Nhiễu Gaussian, còn được gọi là nhiễu ngẫu nhiên có phân bố Gaussian, là một loại nhiễu thường gặp trong xử lý tín hiệu. Nó được xác định bởi hai thước đo chính: xu hướng trung tâm và sự phân tán. Giá trị trung bình (mean) thể hiện vị trí trung tâm của phân phối, trong khi phương sai (variance) cho biết mức độ phân tán của các giá trị xung quanh giá trị trung bình.

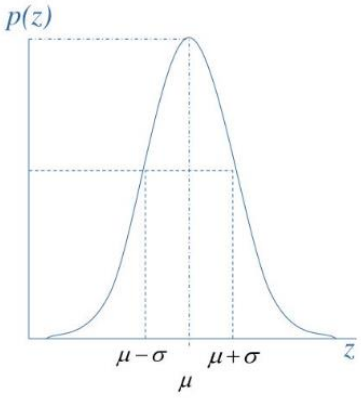
Đặc điểm của phân bố Gaussian là các giá trị thực của các phần tử trong phân bố này đối xứng về giá trị trung bình, và hầu hết các phần tử tập trung ở vùng lân cận của giá trị trung bình. Điều này có nghĩa là các giá trị gần với giá trị trung bình xuất hiện thường xuyên hơn, trong khi các giá trị xa hơn ít xuất hiện hơn.

Phân bố Gaussian được biểu diễn bởi đường cong hình chuông, với đỉnh của đường cong tương ứng với giá trị trung bình và độ rộng của đường cong phản ánh phương sai của phân phối. Nhiễu Gaussian thường được sử dụng trong các mô hình thống kê và lý thuyết thông tin vì tính chất toán học dễ xử lý và khả năng mô hình hóa nhiều hiện tượng tự nhiên.

*b) Công thức tính nhiễu*

Công suất của tín hiệu nhiễu :

$$p(z) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(z-\mu)^2/2\sigma^2}$$




μ : Giá trị trung bình

σ : Độ lệch chuẩn

$\sigma^2$ : Phương sai

c) Đoạn code để tạo nhiễu Gaussian cao tần :

```
# Tạo nhiễu Gaussian
noise = np.random.normal(0, 0.02, audioData.shape) # độ lệch chuẩn là 0.02

# Tạo bộ lọc thông cao (high-pass filter) để tạo nhiễu Gaussian có tần số cao
hp_filter_order = 101
hp_cutoff_frequency = 5000  # Tần số cắt là 5000 Hz
hp_cutoff_normalized = hp_cutoff_frequency / (sampleRate / 2)  # Chuẩn hóa tần
số cắt

# Thiết kế bộ lọc thông cao
b_hp = firwin(hp_filter_order, hp_cutoff_normalized, pass_zero=False)
a_hp = 1

# Áp dụng bộ lọc thông cao lên nhiễu Gaussian
high_freq_noise = lfilter(b_hp, a_hp, noise)

# Thêm nhiễu Gaussian tần số cao vào tín hiệu âm thanh
audioData_noise = audioData + high_freq_noise * np.max(audioData)
```

- Để tạo một tín hiệu nhiễu cao tần, đầu tiên chúng ta cần tạo một bộ lọc thông cao FIR để giữ lại những tín hiệu có tần số cao; bậc lọc là 101 và tần số cắt là 5000Hz.
- Sau đó ta tiếp tục sử dụng hàm có sẵn trong thư viện numpy của python để thiết lập thông số cho nhiễu.

- Dòng mã : “ noise = np.random.normal(0, 0.02, audioData.shape) "tạo ra một mảng nhiễu Gaussian với giá trị trung bình là 0 và độ lệch chuẩn là 0.02, có cùng kích thước với mảng audioData.
- 0: Là giá trị trung bình (mean) của phân bố Gaussian. Điều này có nghĩa là các giá trị ngẫu nhiên sẽ dao động xung quanh 0.
- 0.02: Là độ lệch chuẩn (standard deviation) của phân bố Gaussian. Điều này xác định mức độ phân tán của các giá trị xung quanh giá trị trung bình. Độ lệch chuẩn nhỏ hơn sẽ làm cho các giá trị tập trung gần giá trị trung bình hơn.
- audioData.shape: Là kích thước của mảng audioData. Điều này đảm bảo rằng mảng nhiễu Gaussian được tạo ra có cùng số lượng phần tử như mảng audioData, giúp dễ dàng cộng hai mảng này với nhau.

**3. Thiết kế bộ lọc thông thấp FIR**

*a) Đoạn code lọc thông thấp FIR*

```
# Bộ lọc FIR thông thấp
filterOrder = 201
cutoffFrequency = 4500  # Tần số cắt là 4500 Hz
cutoffNormalized = cutoffFrequency / (sampleRate / 2)  # Chuẩn hóa tần số cắt

# Thiết kế bộ lọc FIR thông thấp
b_lp = firwin(filterOrder, cutoffNormalized, window='hamming')
a_lp = 1

# Áp dụng bộ lọc FIR thông thấp lên tín hiệu có nhiễu
yn = lfilter(b_lp, a_lp, audioData_noise)
```

❖ Giải thích code :

- filterOrder = 201: Số bậc lọc 201. Độ dài bộ lọc càng cao thì đáp ứng tần số của bộ lọc càng chính xác, nhưng cũng làm tăng độ trễ và khối lượng tính toán.
- cutoffFrequency = 4500: Tần số cắt là 4500 Hz. Tín hiệu dưới tần số này sẽ được giữ lại, trong khi tín hiệu trên tần số này sẽ bị suy giảm.
- cutoffNormalized = cutoffFrequency / (sampleRate / 2): Tần số cắt được chuẩn hóa bằng cách chia cho một nửa tần số lấy mẫu (Nyquist rate).
- firwin(filterOrder, cutoffNormalized, window='hamming'): Hàm firwin từ thư viện SciPy thiết kế một bộ lọc FIR thông thấp với:

- filterOrder: Độ dài của bộ lọc.
- cutoffNormalized: Tần số cắt chuẩn hóa.
- window='hamming': Sử dụng cửa sổ Hamming để thiết kế bộ lọc. Cửa sổ Hamming giúp làm mượt đáp ứng tần số của bộ lọc và giảm các hiệu ứng biên.

- b_lp: Hệ số ngõ vào.
- a_lp = 1: Hệ số của bộ lọc ngõ ra (ở đây không có hồi tiếp, nên a_lp bằng 1).

*b) Giải thích lý do chọn cửa sổ Hamming cho thiết kế :*

❖ Cửa sổ Hamming được sử dụng rộng rãi trong thiết kế bộ lọc FIR vì nó có nhiều ưu điểm giúp cải thiện hiệu suất của bộ lọc. Dưới đây là những lý do tại sao nên sử dụng cửa sổ Hamming:

– Giảm thiểu hiệu ứng biên (Spectral Leakage) : Cửa sổ Hamming giúp giảm thiểu hiệu ứng biên, hay còn gọi là spectral leakage, xảy ra khi tín hiệu bị cắt đột ngột tại các biên của cửa sổ. Điều này giúp cải thiện độ chính xác của đáp ứng tần số của bộ lọc, làm cho nó mượt mà hơn.

– Đáp ứng tần số mượt mà : Cửa sổ Hamming tạo ra một bộ lọc có đáp ứng tần số mượt mà hơn so với các cửa sổ hình chữ nhật hoặc các cửa sổ khác. Điều này có nghĩa là các bộ lọc sử dụng cửa sổ Hamming sẽ có sự chuyển tiếp giữa vùng thông và vùng chặn ít gợn sóng hơn, giảm thiểu hiện tượng ripple trong dải tần passband và stopband.

– Giảm biên độ gợn sóng (Ripple) trong vùng thông (Passband) và vùng chặn (Stopband) : Cửa sổ Hamming có khả năng giảm biên độ gợn sóng trong cả vùng thông và vùng chặn. Điều này có nghĩa là tín hiệu trong vùng thông sẽ ít bị biến dạng hơn và tín hiệu trong vùng chặn sẽ bị suy giảm hiệu quả hơn.

– Tính khả thi và dễ sử dụng : Cửa sổ Hamming dễ tính toán và triển khai trong các thuật toán thiết kế bộ lọc. Nó được tích hợp sẵn trong nhiều thư viện xử lý tín hiệu, như SciPy trong Python, làm cho việc sử dụng trở nên thuận tiện và dễ dàng.

**4. Đoạn code hoàn chỉnh cho toàn chương trình :**

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy import signal
from scipy.signal import firwin, lfilter, freqz
import IPython.display as ipd #Thêm thư viện để display Audio
# Đường dẫn đến file audio
filename = 'smallgirl.wav'
# Đọc file audio
sampleRate, audioData = wavfile.read(filename)
# Sử dụng kênh âm thanh đầu tiên nếu là âm thanh đa kênh
audioData = audioData[:, 0] if audioData.ndim > 1 else audioData
# Tạo nhiễu Gaussian
noise = np.random.normal(0, 0.02, audioData.shape) # độ lệch chuẩn là 0.02
# Tạo bộ lọc thông cao (high-pass filter) để tạo nhiễu Gaussian có tần số cao
hp_filter_order = 101
hp_cutoff_frequency = 5000  # Tần số cắt là 5000 Hz
hp_cutoff_normalized = hp_cutoff_frequency / (sampleRate / 2)  # Chuẩn hóa tần
số cắt
# Thiết kế bộ lọc thông cao
b_hp = firwin(hp_filter_order, hp_cutoff_normalized, pass_zero=False)
a_hp = 1
```

```python
# Áp dụng bộ lọc thông cao lên nhiễu Gaussian
high_freq_noise = lfilter(b_hp, a_hp, noise)
# Thêm nhiễu Gaussian tần số cao vào tín hiệu âm thanh
audioData_noise = audioData + high_freq_noise * np.max(audioData)
# Bộ lọc FIR thông thấp
filterOrder = 201
cutoffFrequency = 4500  # Tần số cắt là 4500 Hz
cutoffNormalized = cutoffFrequency / (sampleRate / 2)  # Chuẩn hóa tần số cắt
# Thiết kế bộ lọc FIR thông thấp
b_lp = firwin(filterOrder, cutoffNormalized, window='hamming')
a_lp = 1
# Áp dụng bộ lọc FIR thông thấp lên tín hiệu có nhiễu
yn = lfilter(b_lp, a_lp, audioData_noise)

# Phân tích biên độ tín hiệu
def analyze_amplitude(signal, signal_name):
    max_amplitude = np.max(signal)
    min_amplitude = np.min(signal)
    mean_amplitude = np.mean(signal)
    print(f'{signal_name}:')
    print(f'\tBiên độ tối đa: {max_amplitude}')
    print(f'\tBiên độ tối thiểu: {min_amplitude}')
    print(f'\tBiên độ trung bình: {mean_amplitude}')
    print(f'\tBiên độ RMS: {np.sqrt(np.mean(signal**2))}\n')

# Phân tích biên độ của tín hiệu gốc
analyze_amplitude(audioData, 'Tín hiệu gốc')

# Phân tích biên độ của tín hiệu có nhiễu
analyze_amplitude(audioData_noise, 'Tín hiệu có nhiễu')

# Phân tích biên độ của tín hiệu đã lọc
analyze_amplitude(yn, 'Tín hiệu đã lọc')

# Vẽ FFT và dạng sóng trong miền thời gian của các tín hiệu
plt.figure(1)
plt.plot(np.abs(np.fft.fft(audioData))[:len(audioData)//2])
plt.title('FFT của âm thanh gốc')
plt.xlabel('Tần số (Hz)')
plt.ylabel('Biên độ')

plt.figure(2)
plt.plot(audioData)
plt.title('Tín hiệu âm thanh gốc')
plt.xlabel('Mẫu')
plt.ylabel('Biên độ')

plt.figure(3)
```

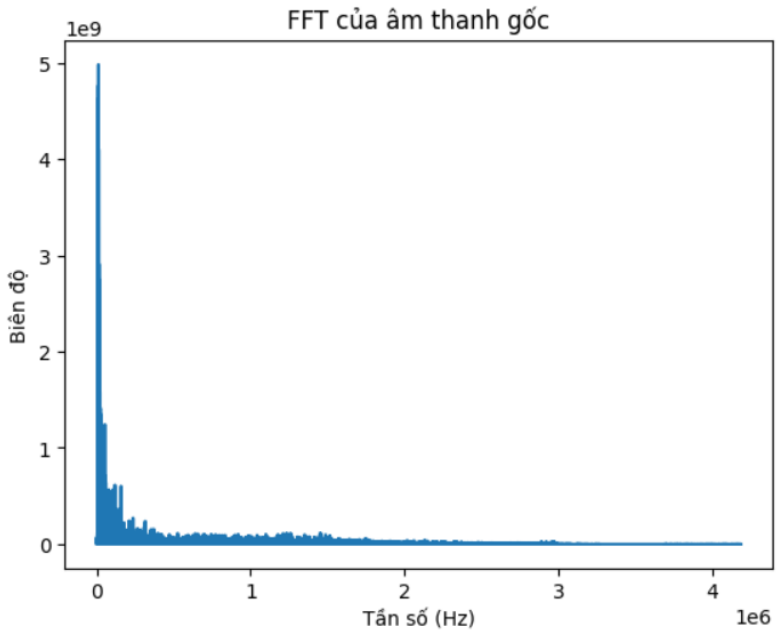
```python
plt.plot(np.abs(np.fft.fft(audioData_noise))[:len(audioData_noise)//2])
plt.title('FFT của âm thanh có nhiễu')
plt.xlabel('Tần số (Hz)')
plt.ylabel('Biên độ')

plt.figure(4)
plt.plot(audioData_noise)
plt.title('Tín hiệu âm thanh có nhiễu')
plt.xlabel('Mẫu')
plt.ylabel('Biên độ')

plt.figure(5)
plt.plot(np.abs(np.fft.fft(yn))[:len(yn)//2])
plt.title('FFT của âm thanh đã lọc')
plt.xlabel('Tần số (Hz)')
plt.ylabel('Biên độ')

plt.figure(6)
plt.plot(yn)
plt.title('Tín hiệu âm thanh đã lọc')
plt.xlabel('Mẫu')
plt.ylabel('Biên độ')

# Hiển thị đáp ứng tần số của bộ lọc thông thấp
w_lp, h_lp = freqz(b_lp, a_lp, worN=9600)
plt.figure(7)
plt.plot(0.5 * sampleRate * w_lp / np.pi, np.abs(h_lp), 'b')
plt.title("Đáp ứng tần số của bộ lọc FIR thông thấp")
plt.xlabel('Tần số (Hz)')
plt.ylabel('Gain')
plt.grid()

# Hiển thị thông tin cơ bản về file âm thanh
print(f'Tần số lấy mẫu: {sampleRate}')
print(f'Số lượng mẫu: {len(audioData)}')
print(f'Thời lượng: {len(audioData) / sampleRate} giây')

# Ghi âm thanh có nhiễu vào file
noisy_filename = 'noisy.wav'
wavfile.write(noisy_filename, sampleRate, audioData_noise.astype(np.int16))

# Ghi âm thanh đã lọc vào file
filtered_filename = 'filtered.wav'
wavfile.write(filtered_filename, sampleRate, yn.astype(np.int16))
plt.show()
```

# Chương IV : Kết quả của tín hiệu

## 1. Các phổ tín hiệu của các file âm thanh :

Biểu đồ phổ Fourier (FFT - Fast Fourier Transform) của một âm thanh.

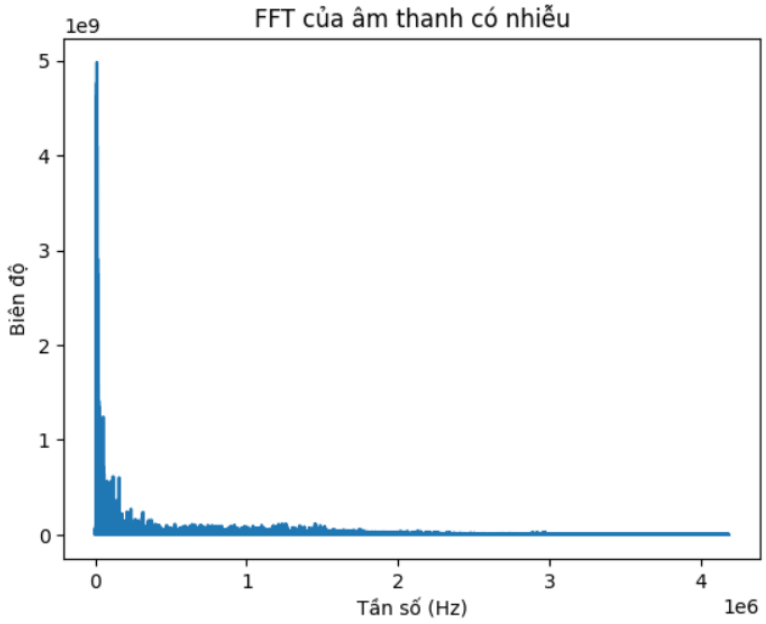
- Trục hoành (x-axis): Tần số (Hz)
- Trục tung (y-axis): Biên độ




*Phổ tín hiệu âm thanh gốc*

Biểu đồ này cho thấy cách biên độ của các tần số khác nhau trong tín hiệu âm thanh thay đổi. Dựa trên biểu đồ, có thể thấy rằng:

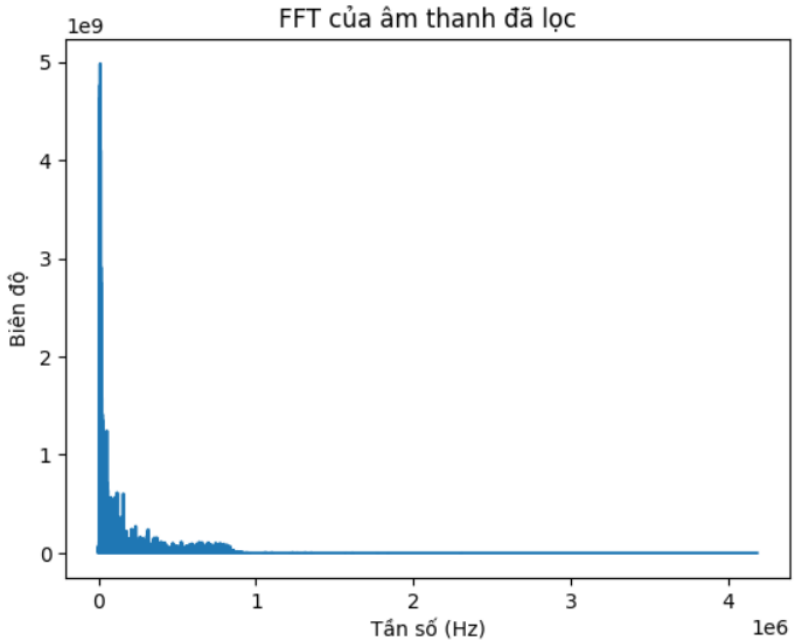
- Tín hiệu có một đỉnh biên độ rất cao ở tần số rất thấp, cho thấy tín hiệu này chứa nhiều thành phần tần số thấp.
- Sau đó, biên độ giảm dần và có một vài đỉnh nhỏ hơn ở các tần số cao hơn.
- Tần số càng cao, biên độ càng giảm và gần như bằng không ở các tần số rất cao.
- Điều này gợi ý rằng âm thanh gốc chủ yếu bao gồm các thành phần tần số thấp và ít có các thành phần tần số cao.




*Phổ tín hiệu âm thanh có nhiễu*

Biểu đồ này cho thấy cách biên độ của các tần số khác nhau trong tín hiệu âm thanh thay đổi. Dựa trên biểu đồ, có thể thấy rằng:

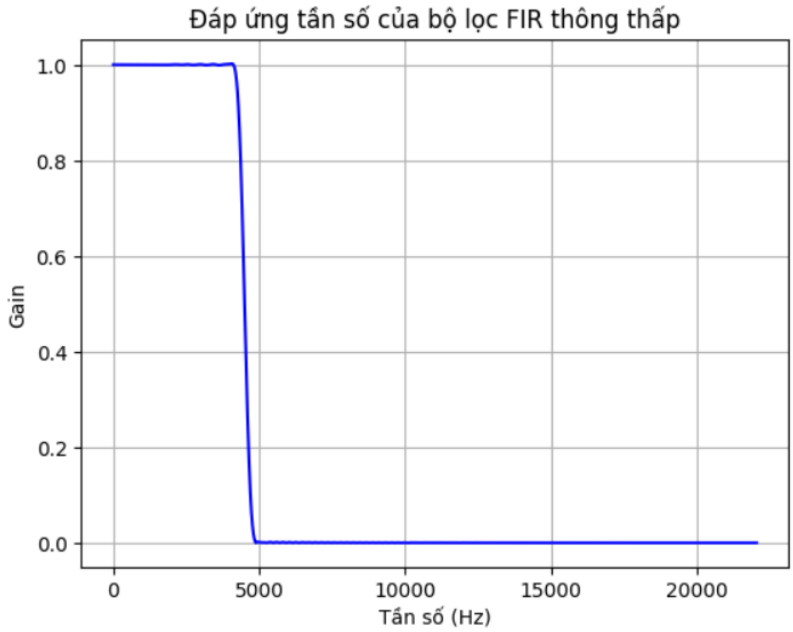
– Phổ âm thanh có nhiễu không khác quá nhiều so với phổ tín hiệu âm thanh gốc, nhưng nếu để ý kỹ thì sẽ thấy được ở những tần số cao có biên độ cao hơn so với biên độ âm thanh gốc. Điều này xảy ra do tác động của nhiễu Gaussian cao tần đã được thiết kế ở trên, do độ lệch chuẩn của nhiễu chỉ có 0.02 nên sự thay đổi khó nhận thấy bằng mắt được.




*Phổ tín hiệu âm thanh đã lọc*

Biểu đồ này cho thấy cách biên độ của các tần số khác nhau trong tín hiệu âm thanh thay đổi. Dựa trên biểu đồ, có thể thấy rằng:

– Sau khi đã lọc thì các tín hiệu âm thanh cao tần đã được loại bỏ khỏi phổ tín hiệu âm thanh, âm thanh khi đã lọc sẽ không còn nhiễu nhưng tín hiệu sẽ không được to rõ như tín hiệu gốc vì tần số cắt của tín hiệu chỉ 4500Hz




*Đáp ứng tần số khi đã lọc FIR thông thấp*

**2. Kết quả tính toán từ đoạn code :**

Tần số lấy mẫu (Sampling Frequency): 44100 Hz (chuẩn CD chất lượng âm thanh).

Số lượng mẫu (Number of Samples): 8,369,152.

Thời lượng (Duration): 189.7767 giây.

- Tín hiệu gốc:
– Biên độ tối đa (Max Amplitude): 32767
– Biên độ tối thiểu (Min Amplitude): -32768
– Biên độ trung bình (Mean Amplitude): -0.1863
– Biên độ RMS (Root Mean Square Amplitude): 9.1118

➔ Tín hiệu gốc có biên độ nằm trong khoảng -32768 đến 32767, đây là biên độ chuẩn của tín hiệu âm thanh 16-bit. Biên độ trung bình rất gần với 0, cho thấy tín hiệu không bị lệch đáng kể. Biên độ RMS khá thấp, cho thấy tín hiệu gốc có mức năng lượng thấp.

- Tín hiệu có nhiễu:
– Biên độ tối đa (Max Amplitude): 34355.1459
– Biên độ tối thiểu (Min Amplitude): -34400.3254

- Biên độ trung bình (Mean Amplitude): -0.1865
- Biên độ RMS (Root Mean Square Amplitude): 8507.9788

➔ Tín hiệu có nhiễu có biên độ tối đa và tối thiểu lớn hơn so với tín hiệu gốc, cho thấy có thêm nhiễu với biên độ cao. Biên độ trung bình vẫn rất gần với 0, không bị lệch nhiều. Tuy nhiên, biên độ RMS tăng đáng kể, cho thấy tín hiệu có nhiễu có mức năng lượng cao hơn.

- Tín hiệu đã lọc:
- Biên độ tối đa (Max Amplitude): 33203.1095
- Biên độ tối thiểu (Min Amplitude): -33687.8497
- Biên độ trung bình (Mean Amplitude): -0.1864
- Biên độ RMS (Root Mean Square Amplitude): 8394.9012

➔ Tín hiệu đã lọc có biên độ tối đa và tối thiểu giảm so với tín hiệu có nhiễu nhưng vẫn lớn hơn tín hiệu gốc. Điều này cho thấy bộ lọc đã giảm bớt nhiễu nhưng vẫn còn một số thành phần nhiễu. Biên độ trung bình vẫn rất gần với 0. Biên độ RMS giảm so với tín hiệu có nhiễu, cho thấy bộ lọc đã làm giảm mức năng lượng của nhiễu nhưng vẫn còn cao hơn tín hiệu gốc.

❖ Các file âm thanh :

do_an4\smallgirl.wav

do_an4\noisy.wav

do_an4\filtered.wav

## Chương V : Kết luận

Việc xử lý tín hiệu âm thanh bằng bộ lọc FIR thông thấp đóng một vai trò quan trọng và mang lại nhiều lợi ích thiết thực trong cuộc sống. Tín hiệu gốc, với biên độ nằm trong phạm vi chuẩn của tín hiệu âm thanh 16-bit và mức năng lượng thấp, thường mang lại chất lượng âm thanh rõ ràng và trung thực. Tuy nhiên, trong môi trường thực tế, tín hiệu này thường bị ảnh hưởng bởi các yếu tố nhiễu bên ngoài, dẫn đến mức năng lượng tổng thể của tín hiệu bị tăng lên do sự có mặt của các thành phần nhiễu không mong muốn.

Khi áp dụng bộ lọc FIR thông thấp, chúng ta nhận thấy rằng mức năng lượng của tín hiệu nhiễu đã được giảm bớt một cách đáng kể. Bộ lọc FIR có khả năng loại bỏ các tần số cao không mong muốn, giữ lại phần tín hiệu gốc có giá trị. Điều này giúp cải thiện chất lượng âm thanh và giảm thiểu sự khó chịu do nhiễu gây ra. Tuy nhiên, mặc dù bộ lọc đã thể hiện hiệu quả rõ rệt, mức năng lượng sau khi lọc vẫn còn cao hơn so với tín hiệu gốc. Điều này chỉ ra rằng, mặc dù bộ lọc FIR có thể cải thiện chất lượng tín hiệu, nhưng không thể hoàn toàn loại bỏ tất cả các thành phần nhiễu.

Từ quan sát này, chúng ta có thể rút ra một kết luận quan trọng: bất kỳ thiết kế bộ lọc nào cũng chỉ có thể đạt được hiệu quả tương đối, không thể đạt được mức tuyệt đối. Hiệu suất của bộ lọc phụ thuộc vào nhiều yếu tố, bao gồm cấu trúc của bộ lọc, đặc tính của tín hiệu và mức độ nhiễu. Không có một giải pháp nào có thể hoàn toàn loại bỏ được nhiễu mà không ảnh hưởng đến tín hiệu gốc ở một mức độ nào đó.

Điều quan trọng nhất để đạt được kết quả lọc tốt là đảm bảo rằng dữ liệu đầu vào càng ít nhiễu càng tốt. Việc giảm thiểu nhiễu từ nguồn gốc không chỉ làm cho quá trình lọc trở nên dễ dàng hơn mà còn giúp duy trì chất lượng tín hiệu gốc. Điều này có thể đạt được thông qua việc sử dụng các phương pháp che chắn chống nhiễu, chọn lựa các thiết bị thu tín hiệu chất lượng cao và đặt chúng trong môi trường ít nhiễu.

Trong tương lai, với sự phát triển của công nghệ và các thuật toán xử lý tín hiệu tiên tiến, chúng ta hy vọng sẽ có những bộ lọc có hiệu suất cao hơn, giúp cải thiện chất lượng âm thanh và tín hiệu một cách đáng kể. Tuy nhiên, luôn luôn nhớ rằng, việc kiểm soát và giảm thiểu nhiễu từ nguồn gốc là yếu tố then chốt để đạt được kết quả lọc tối ưu.